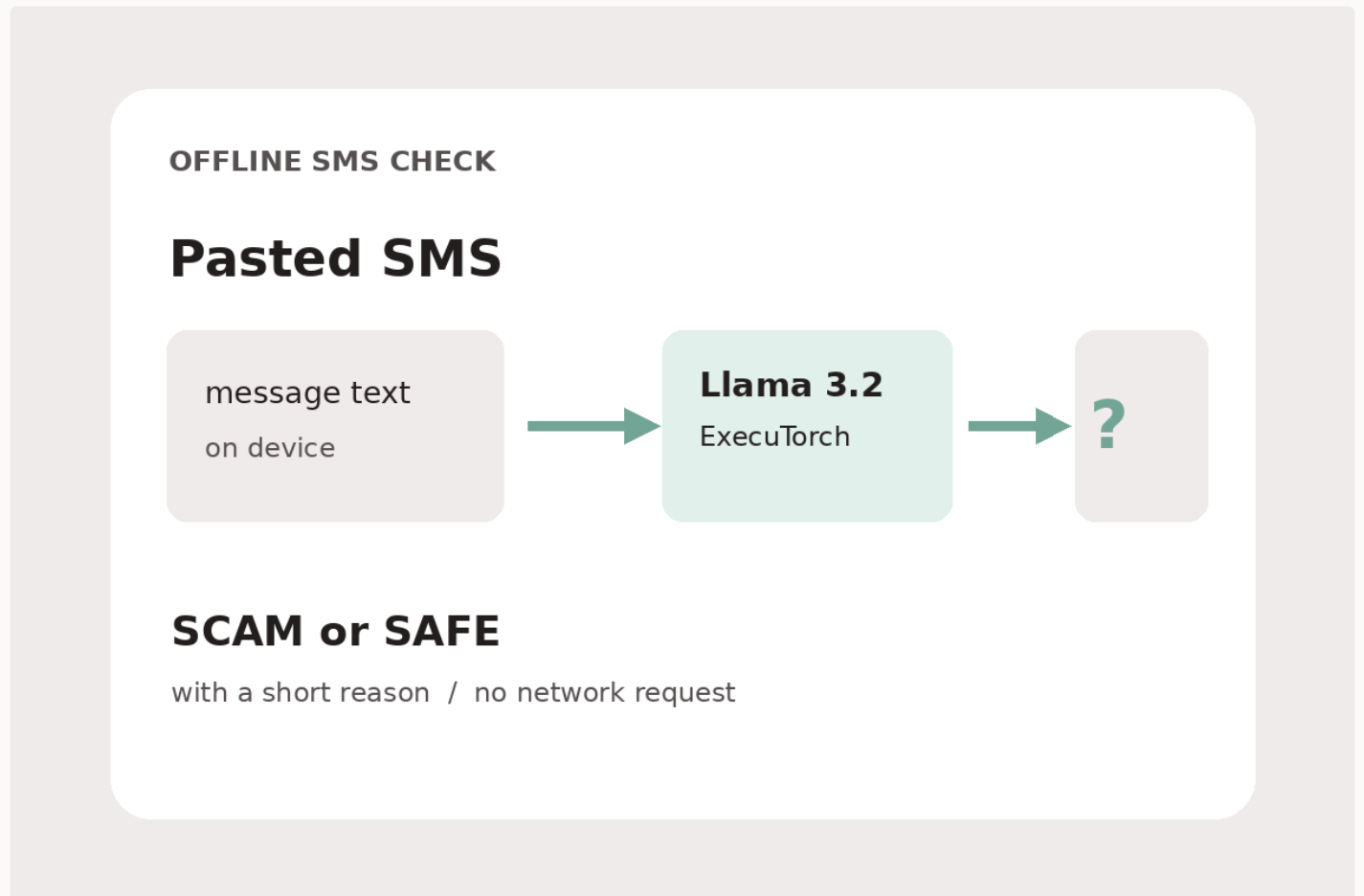


UnScam: Technical Report

Offline on-device SMS scam detector

Our offline Android prototype checks pasted SMS for scam signs and gives a short reason for its verdict. This report records the working prototype and its current evidence, while clearly separating emulator results from future physical-device tests.

At a glance



Concept diagram. This is not a screenshot of the app.

1. Abstract

We built UnScam, an Android app that lets a person paste an SMS and receive a SCAM or SAFE label followed by a one-line explanation. The model is Llama-3.2-1B-Instruct, adapted with LoRA and exported to ExecuTorch for local inference on an Arm-based Android target. The app does not need an internet connection to classify a message. On a 164-message held-out split from the same dataset family, the fine-tuned model answered 162 correctly (98.8%). That result is an in-distribution test, not a claim of 98.8% accuracy on all real Indian SMS.

2. Problem and goal

Scam SMS can imitate banks, delivery services, employers, and government services. A label alone is not enough: a user should understand which part of the message looks risky. Our goal is a simple tool that runs locally and explains its decision in plain language. We target messages pasted by the user, rather than claiming to inspect the phone's inbox automatically. The prototype focuses on English-language SMS.

We chose a Llama-family model for the explanation and ExecuTorch for local deployment on Android. We use the forked ExecuTorch LlamaDemo as the Android app base, not a cloud inference service.

3. System design

From input to answer

- The user pastes one SMS into the app.
- The app formats a fresh Llama 3 conversation with a fixed system instruction and that SMS as the user message.
- The locally loaded model generates SCAM or SAFE, followed by a one-line reason. Temperature is 0 for repeatable verdicts.
- The app displays the result; no network request is needed for inference.

The fixed instruction is: "You are an SMS scam detector for Indian users. Reply SCAM or SAFE, then one line why." The Llama-3 chat-template headers use two newlines before message text, matching the fine-tuning format. We aligned this in `PromptFormat.kt`, corrected the system-prompt punctuation, set the model type to `LLAMA_3`, and set `maxSeqLen` to 512. These formatting details matter because an otherwise valid model can produce poor answers if training and app prompts differ.

On-device runtime

We exported the fine-tuned model with ExecuTorch 1.1.0 using `export_llm`, `llama_xnnpack.yaml` and overrides, then applied int4 quantization with torchao 0.15.0. The resulting .pte file is about 765 MB. Inference uses the XNNPACK backend on an Arm CPU, with a maximum context of 512 tokens. On the Pixel 8 Pro emulator, the model loads in about 1.9 seconds and answers in a few seconds. These are observations from the emulator, not benchmark results from a physical phone.

4. Data and training

Our working dataset has 1,638 labeled SMS: 1,087 SAFE and 551 SCAM. We seeded it from the public SMS Phishing Dataset by Sandhya Mishra and Devpriya Soni (Mendeley Data, 2022), then added Indian-context examples from collected specimens and a small compiled set. The public source is larger than our selected working set; its original categories are not identical to our binary SCAM/SAFE labels. We still need to document the exact selection and label mapping in the repository for reproducibility. The current working set is not wholly Indian, and its Indian subset has not been separately counted here.

The Indian-context examples include KYC or account-block claims, electricity-disconnection threats, fake jobs, KBC lottery claims, parcel/customs fees, income-tax refunds, UPI fraud, and bank-suspension claims. Legitimate messages, including routine alerts, are important so that the app does not flag every urgent-looking notice as a scam.

We split the working set into 1,474 training messages and 164 held-out messages. We fine-tuned Llama-3.2-1B-Instruct on a free Colab GPU using Hugging Face Transformers and PEFT, with fp16 and LoRA rank 16. The run took 186 training steps; reported loss fell from 2.31 to about 0.005. These training-loss numbers describe fit to the training task, not real-world reliability.

5. Evaluation and demonstration

On the held-out split, 162 of 164 messages were classified correctly: 98.8% accuracy. The split comes from the same overall dataset and message style used for training. We therefore do not treat that number as an independent field test or a guarantee across new scams, languages, or senders.

In live app trials on the Pixel 8 Pro emulator, scam-link examples returned SCAM with relevant reasons; OTPs, delivery notifications, and personal messages returned SAFE. These are qualitative checks rather than a controlled, counted benchmark. Our 61-second demo video shows the app loading its model, classifying an SBI scam message with a reason, and treating a routine shopping message as SAFE. The video and measurements are from an emulator, not a physical Android phone.

What we can claim now

We have a working offline prototype and a promising same-dataset evaluation. We have not yet validated performance, latency, or memory on a physical Arm Android device, nor measured accuracy on a separately sourced Indian SMS test set.

6. Limits and responsible use

- The test set resembles the training data; a new sender or scam style may be harder.
- The 1,638-message working set is small, mixed in origin, and has limited Indian examples.
- The present prototype focuses on English; Hindi, Hinglish, and regional-language messages need dedicated evaluation.
- The 512-token context limit means long pasted text can be truncated.
- All current device testing was on an emulator; physical Arm hardware testing remains open.
- A SCAM or SAFE label is advice, not proof. A user should independently verify a sensitive request through an official channel.

7. Next steps

- Run the same test and profile latency, memory, and offline behavior on a physical Arm Android phone.
- Add more real, responsibly sourced Indian scam and legitimate examples; record provenance and test on a separately sourced set.
- Extend and test support for Hindi, Hinglish, and regional-language SMS.
- Explore on-device SMS inbox reading with clear consent and permission handling; it is not part of the current prototype.

8. Project files and reference

Source code: [UnScam's ExecuTorch examples fork](#). The merged model is stored in the private Hugging Face repository [siddi11/unscam-model](#); access is not public. The 61-second demonstration video is included in the repository at [demo.mp4](#). The application is demonstrated on an Android emulator. The packaged app artifact has not yet been added to the repository.

Public dataset origin: [Mishra and Soni, SMS Phishing Dataset, Mendeley Data \(2022\)](#).

Prepared 26 September 2026. Physical-device results and a separately sourced field evaluation remain future work.